ChartVoice team brief

Oct 9, 2026 ·

ChartVoice turns a chart image into an accurate data table, a plain-language summary and sound, so blind, low-vision and chart-averse readers get the actual numbers. We build it at Banana Hacks 2026 (Oct 10, 5 AM to Oct 12, 9 AM GMT+5) and submit by Sunday evening.

The problem and who it is for

Charts carry the key numbers in textbooks, papers, news and financial reports, but they are images. A screen reader usually says only "image" or reads a vague alt text, so the data is lost.

User group	What goes wrong today	What ChartVoice gives them
Blind users (screen readers)	The chart is silent or described vaguely	A navigable data table, spoken summary and sound
Low-vision users	Small labels, thin lines and low contrast are hard to read	A large, high-contrast redrawn chart and large-text table
People who find charts hard to read (low data literacy, dyslexia, unfamiliar chart types)	They misread scales or miss the main trend	A plain-language summary: highest, lowest, trend, biggest change

One-line pitch: "ChartVoice lets anyone hear, read and check the real numbers inside a chart image."

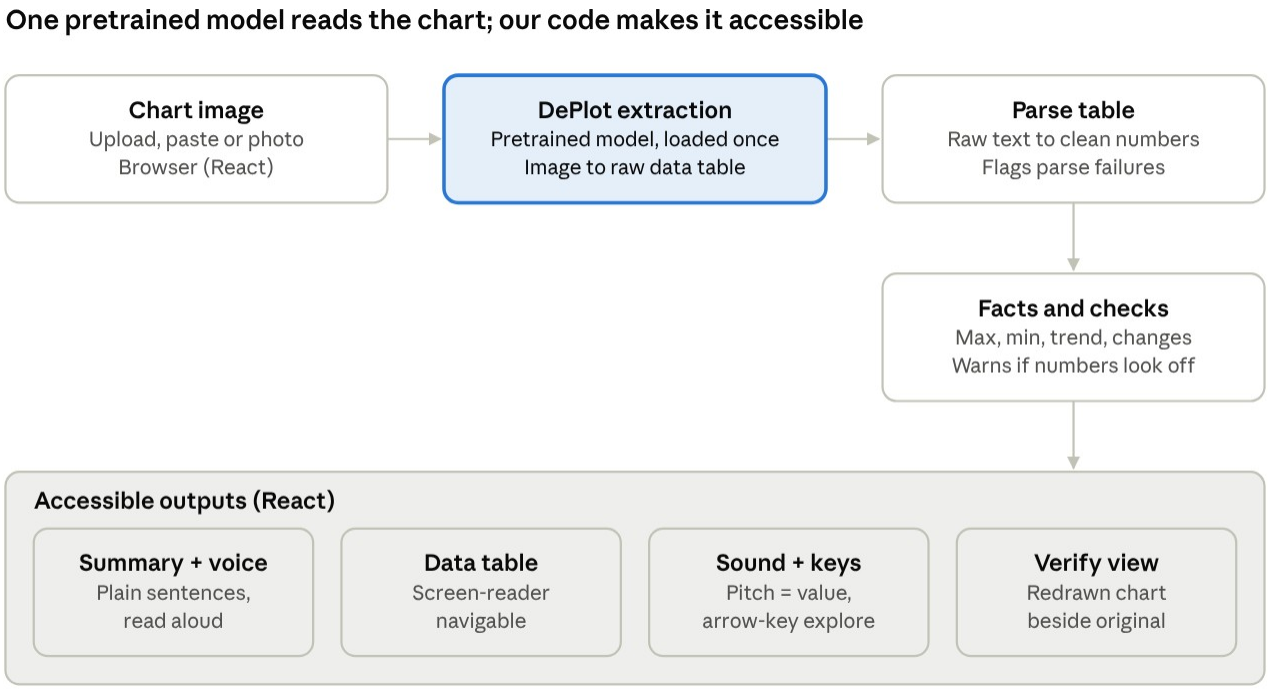
Features

F1 to F6 are the must-have core; F7 to F10 are added Saturday night and Sunday if the core works. Scope is bar, line and pie charts only.

#	Feature	What the user gets	Priority
F1	Chart input	Upload a screenshot, paste from clipboard, take a phone photo, or pick a sample chart	Must

F2	Data extraction	The chart's underlying data table, read from the image by the DePlot model	Must
F3	Facts engine	Highest, lowest, overall trend, biggest change, share of total (pie) computed in Python, not guessed	Must
F4	Plain-language summary	3 to 5 short sentences built from F3's facts, e.g. "Sales rose from 20 in January to 55 in June; the biggest jump was April to May (+15)."	Must
F5	Read aloud	The summary spoken with the browser's speech engine	Must
F6	Accessible data table	A real HTML table a screen reader can navigate cell by cell	Must
F7	Sonification	The chart played as tones: higher value = higher pitch, left to right over time	Should
F8	Keyboard explorer	Arrow keys move point by point; each value is announced, e.g. "April, 40"	Should
F9	Verification view	The extracted data redrawn as a large, highcontrast chart next to the original, so a sighted helper can check it at a glance	Should
F10	Trust warnings	Flags when extraction looks wrong, e.g. pie shares that do not sum to about 100%, or a table that failed to parse	Should
F11	Comparison mode	A generic AI description of the same chart next to ChartVoice's output, for the demo and evaluation	Nice to have

How it works



ChartVoice pipeline · image to accessible outputs

Only the DePlot step uses an AI model; parsing, facts, checks and every output are our own code, which is where our contribution lies. A request takes a few seconds on a laptop CPU, with no training.

Functions

No model training: we use pretrained weights, load them once when the server starts, and run only fast inference per request. Everything runs on a normal laptop CPU; a Colab T4 is optional for batch evaluation.

Backend (Python + FastAPI)

File	Function	Input to output
`models.py`	`load_models()`	Runs once at startup; loads DePlot (`google/deplot` , Apache-2.0) into memory
File	Function	Input to output

`extract.py`	`extract_table(image)`	Chart image to raw linearized table text from DePlot
`parse.py`	`parse_table(raw)`	Raw text to a clean pandas DataFrame: title, column names, numeric values; reports parse failures
`facts.py`	`compute_facts(df, chart_type)`	DataFrame to facts: max, min, trend direction and slope, biggest change, shares of total
`checks.py`	`sanity_checks(df, facts)`	Trust warnings: missing values, nonnumbers, pie not summing to about 100%
`summary.py`	`build_summary(facts)`	Facts to 3 to 5 plain-language template sentences; no language model, so no invented numbers
`classify.py`	`chart_type(image)`	Optional: bar, line or pie via zero-shot CLIP; otherwise the user picks
`baseline.py`	`generic_description(image)`	Optional: a generic AI description for comparison mode (run on Colab)
`main.py`	`GET /health` , `POST /analyze`	API; `/analyze` returns table, facts, summary, warnings and timing as JSON
`evaluate.py`	`run_eval()`	Compares extracted values with ground-truth tables; prints accuracy and latency

Frontend (React + Vite)

Component	Job
`ChartInput`	Upload, paste, camera and sample charts; Analyze button with loading state
`SummaryPanel`	Summary text with a Read aloud button (`speechSynthesis`)

Component	Job
`DataTable`	Accessible HTML table with caption and header cells
`SoundPlayer`	Sonification with the Web Audio API: value to pitch (about 220 to 880 Hz), one tone per point
`KeyboardExplorer`	Arrow-key navigation; values announced through an `aria-live` region
`VerifyView`	Original image beside the redrawn high-contrast chart (Chart.js)
`WarningBanner`	Shows trust warnings in plain words
`ComparePanel`	Generic AI description vs ChartVoice, side by side

Why not just ChatGPT, and how we prove it

A chatbot can describe a chart, but it can get numbers wrong, and a blind user cannot see the chart to catch the mistake. ChartVoice gives output that can be checked: a table, a redrawn chart, sound and warnings.

	Generic AI description	ChartVoice
Output	A paragraph of text	Table, summary, sound, keyboard exploration, redrawn chart
Numbers	Written by the language model	Extracted from the image, then computed in code
Checkable by a helper	Hard	Yes: original beside redrawn chart
Flags its own errors	Rarely	Sanity-check warnings
Cost and privacy	Paid API, image sent to a third party	Free, runs locally or on our own server

Evaluation plan

We measure; we never invent numbers.

1. Test set: 30 bar, line and pie charts with known data tables (ChartQA's chart images and source tables; check and credit the license in the README). Pick them before tuning anything.
2. Metric 1, value accuracy: share of extracted values within 5% of the true value.
3. Metric 2, summary correctness: is the reported highest, lowest and trend right? Yes or no per chart.
4. Baseline: an open AI model on Colab asked to describe the same charts; score its numbers the same way.
5. Metric 3, latency: median seconds per chart on a laptop CPU.
6. Failure log: 3 to 5 examples where ChartVoice was wrong, shown honestly on the evaluation page.

CV line once measured: "ChartVoice extracted chart values within 5% on X% of 30 test charts vs Y% for a generic AI description."

Team roles

Four roles; with two people, one takes ML + backend and the other frontend + evaluation and video.

Role	Owns	Files
ML and extraction	DePlot running, parsing its output, speed	`models.py` , `extract.py` , `parse.py` , `classify.py`
Backend logic and API	Facts, checks, summary, FastAPI endpoints	`facts.py` , `checks.py` , `summary.py` , `main.py`
Frontend and accessibility	All React components, read aloud, sound, keyboard, screenreader testing	`frontend/`
Evaluation and story	Test charts, ground truth, `evaluate.py` , baseline on Colab, video, Devpost write-up, process notes	`evaluate.py` , `eval/` , `README.md`

Schedule

We work Saturday from 9 AM into the night and Sunday from 11 AM to evening, and submit Sunday by 8 PM because Monday is unavailable (official deadline Monday 9 AM GMT+5).

Saturday Oct 10

- [ ] 9 to 11 AM, feasibility gate: DePlot extracts tables from 10 sample charts (bar, line, pie). Mostly right: go. Poor: switch to ClearPath (same stack)
- [ ] 9 to 11 AM, in parallel: repo skeleton pushed, React + Vite app created, FastAPI `/health` running
- [ ] 11 AM to 2 PM: `extract` , `parse` , `facts` working; `/analyze` returns JSON
- [ ] 2 to 5 PM: summary, checks; frontend shows table and summary from the real API
- [ ] 5 to 9 PM: read aloud, sonification, keyboard explorer
- [ ] 9 PM onward: verification view, warnings, demo-mode saved results for the video End-of-day
- [ ] check: chart image in, spoken summary and sound out
- [ ]

Sunday Oct 11

- [ ] 11 AM to 1 PM: run evaluation on 30 charts plus the baseline on Colab; build the evaluation page
- [ ] 1 to 3 PM: fix worst failures, polish the UI, test with a screen reader (NVDA on Windows is free)
- [ ] 3 to 5 PM: record the video (3 minutes or less), upload to YouTube as public
- [ ] 5 to 7 PM: README and Devpost write-up (inspiration, brainstorming, design process, resources, final product)
- [ ] By 8 PM: submit on Devpost; check the video link and GitHub link work

3-minute video plan

The 3-minute limit is strict; aim for 2:50.

Time	Show	Say
0:00 to 0:20	A screen reader reading a news chart as just "image"	The problem: charts lock data away from blind, low-vision and chartaverse readers
Time	Show	Say
0:20 to 1:30	Live demo: upload chart, table appears, summary read aloud, play the sound, arrow through points	What ChartVoice does
1:30 to 2:00	Pipeline diagram	How it works: DePlot extraction, facts engine, checks, accessible output

2:00 to 2:35	Generic AI vs ChartVoice; evaluation table; one failure case	Why it is trustworthy, with measured numbers
2:35 to 2:50	Verify view and warnings	Future work: PDFs, more chart types, a browser extension

Risks and fallback

Risk	Plan
DePlot is poor on our charts	Saturday 11 AM gate: switch to ClearPath, which uses the same FastAPI + React stack
DePlot is slow on CPU	Resize images first; use demo-mode saved results in the video; Colab T4 for evaluation
Parsing DePlot output breaks	Robust parser with tests on 10 real outputs; show a clear warning instead of crashing
Complex charts (stacked, multi-axis)	Out of scope; say so in the UI and README
Wrong numbers mislead users	Trust warnings, verification view, and a limitations note
Running out of time	Cut F11, then F9; never cut F1 to F6, the evaluation or the video
Rules breach	No code before Oct 10, 5 AM GMT+5; no code from AgriBoost or smartEVM

Setup to do today

Setup and planning are fine before the start; project code is not.

- [ ] Install Python 3.11, Node.js LTS, Git and VS Code
- [ ] Free about 5 GB of disk space for PyTorch and model weights
- [ ] Create an empty GitHub repo `chartvoice` and invite teammates (first commit after 5 AM Saturday)

- [ ] Check Colab gives a T4 GPU (Runtime, Change runtime type)
- [ ] Create a free Hugging Face account for later deployment
- [ ] Install NVDA (free screen reader) for accessibility testing
- [ ] Start a shared notes doc for brainstorming and design decisions; the write-up must cover them
- [ ] Collect 15 to 20 open-license sample charts (bar, line, pie) for the demo
- [ ] Agree on roles and register the team on Devpost
- [ ]